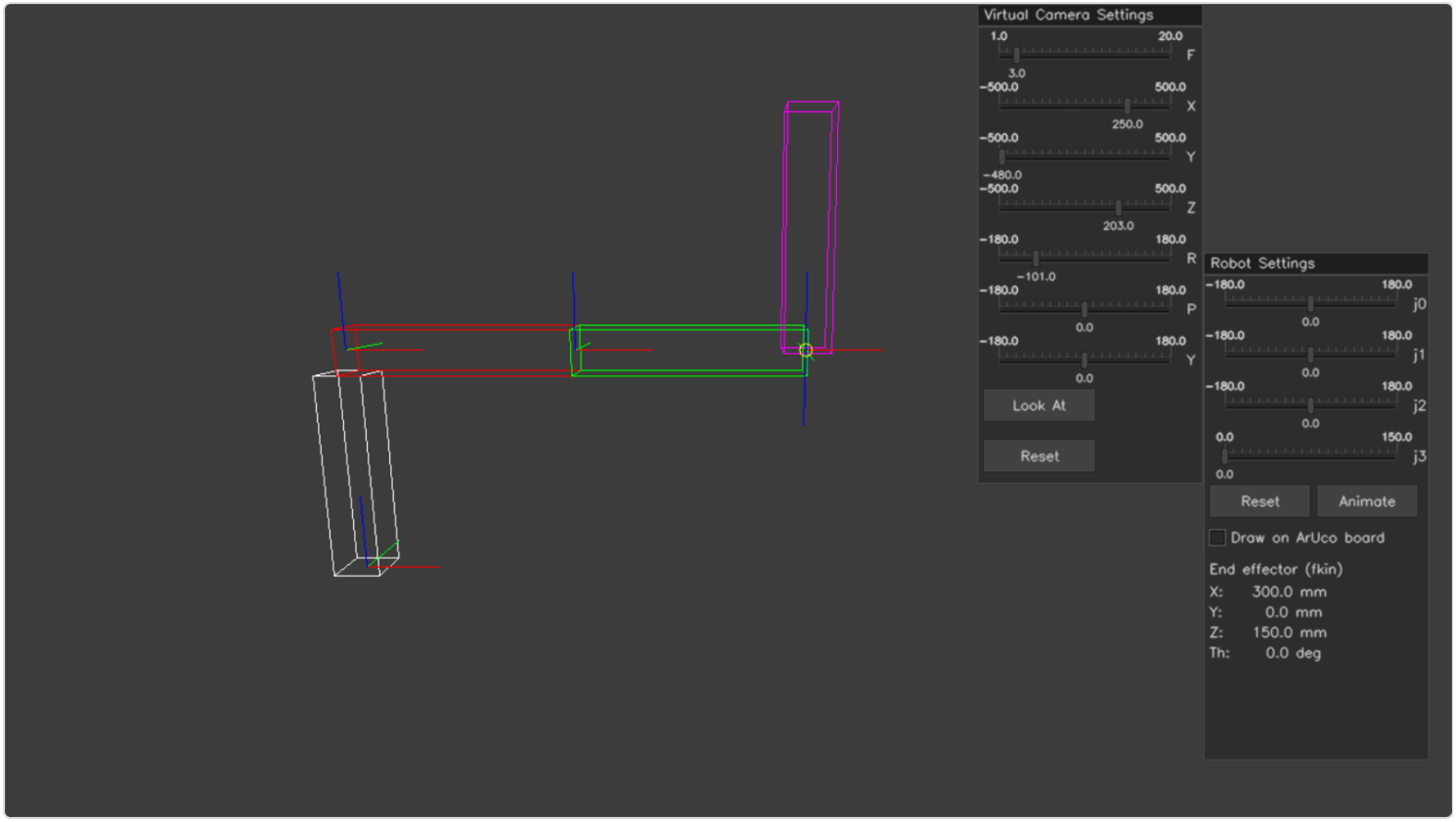
Lab 5 – Forward Kinematics (SCARA): Summary of Changes

Branch: **Lab5_C_Experiment** · ELEX 7825 · Built and run-tested on Visual Studio 18, Release x64.

1. What was done, in one paragraph

A 4-joint SCARA robot (3 revolute + 1 prismatic) was added to `CRobot`, following the same create / draw pattern as the Lab 3 and Lab 4 simple robots. Menu option **(5) Lab 5** now builds the SCARA with `create_scara()` and draws it every frame with `draw_scara()`. A "Robot Settings" window on the right has a slider per joint, Reset, Animate, a live end-effector pose read-out computed by `fkin()`, and a **Draw on ArUco board** checkbox that switches from the virtual camera to the webcam + ChArUco board. Everything starts on the virtual camera, so it can be used without a webcam.



Lab 5 at the home pose (all joints 0). White = pedestal, red = link 1, green = link 2, magenta = quill (link 3). The yellow circle is where fkin() says the tool is.

2. How to run it

- Build and run in Visual Studio as before, then type **5** at the menu.
- Drag **j0, j1, j2** (degrees, −180…180) and **j3** (prismatic, 0…150 mm). The robot and the End effector (fkin) read-out update every frame.
- **Reset** puts all joints back to 0 and stops an animation. **Animate** runs the Part B-A sequence.
- The Virtual Camera Settings window works as in Lab 3; its Reset now returns to the SCARA view.
- Tick **Draw on ArUco board** to use the webcam (Part B-B). Untick it to go back to the virtual camera.
- Press **q** in the window to return to the menu.

3. Files changed

File	Change
Robot.h	Lab 5 members (_scara_links, _scara_colors, _joint[4], _anim_count, _draw_on_board) and Lab 5 functions, each labelled Part A-A / A-B / A-C / B-A / B-B.
Robot.cpp	New "LAB5 - SCARA robot" section at the end: create_scara, scara_transforms, fkin, get_joints, update_scara_settings, animate_scara, draw_scara. init() now also zeroes the joints. The constructor sets _draw_on_board = false.
CameraVirtual.h/.cpp	Added set_default_view(). The Lab 3 default view moved into _default_* members (same numbers as before), so the Reset button can return to whichever view is the default.
7825_Template.cpp	lab5() now calls robot.create_scara() once, then robot.draw_scara() in the loop.
docs/	This summary in HTML, PDF and Word, plus two screenshots.

Unchanged: Lab 3 and Lab 4 code paths (create/draw_simple_robot, *_lab4, update_settings, draw), CameraReal.

4. Robot geometry and coordinate frames

Units are metres and degrees, the same as the Lab 3 boxes. Every link box is 15 cm long and 3 × 3 cm in cross-section. The pedestal is 13.5 cm tall (15 cm − ½ × 3 cm), so the link centre line and the prismatic joint at 0 sit at z = 15 cm. Every transform is built with the existing `createHT(t, r)` (r = rotation about x, y, z in degrees, R = Rz·Ry·Rx).

Frame	Built from parent by	Meaning
wT0	identity	World origin, base of pedestal
T01	createHT((0, 0, 0.15), (0, 0, j0))	Up to the top of the pedestal, rotate j0 about z
T12	createHT((0.15, 0, 0), (0, 0, j1))	Out to the end of link 1, rotate j1 about z
T23	createHT((0.15, 0, 0), (0, 0, j2))	Out to the end of link 2, rotate j2 about z (wrist)
T34	createHT((0, 0, −j3), (180, 0, 0))	Slide down by j3, flip 180° about x so tool z points down

The world poses are chained: wT1 = wT0·T01, wT2 = wT1·T12, wT3 = wT2·T23, wT4 = wT3·T34. wT4 is the end effector.

Each box is stored in the frame of the joint that drives it. The box origin is the centre of the face at that joint, as the lab asks:

Box	createBox(w, h, d)	Offset inside its frame	Drawn with
Pedestal (white)	0.03, 0.03, 0.135	+0.0675 z (sits on z = 0)	wT0
Link 1 (red)	0.15, 0.03, 0.03	+0.075 x	wT1
Link 2 (green)	0.15, 0.03, 0.03	+0.075 x	wT2
Quill (magenta)	0.03, 0.03, 0.15	−0.075 z (tool z is down, so it sticks up)	wT4

5. Part A — functions

Part A-A: create_scara() and draw_scara()

`create_scara()` follows the shape of `create_simple_robot()`: it builds boxes with `createBox()` and moves them with `transformPoints()`. The difference is that the SCARA moves, so the boxes are left in their own joint frames rather than baked into world coordinates. It also sets the colours, resets the joints and calls `_virtualcam.set_default_view(3, 250, -480, 203, -101, 0, 0)` so the whole arm is in view. (This is the view from the lab PDF screenshot, shifted right to clear the UI.)

`scara_transforms(joints)` returns all five world frames {wT0 … wT4} for the current joints. That is the "intermediate joint poses" calculation the lab describes.

`draw_scara()` runs every frame:

- Clears the canvas to grey. In board mode it also pastes the webcam image and runs `detect_markers()`.
- Gets the joint frames from `scara_transforms(get_joints())`.
- For each box: **clones** its points, transforms them by base·wT(frame), then calls `drawBox()`.
- Draws coordinate axes (`createCoord()` + `drawCoord()`) at all five frames.
- Draws a yellow circle at the `fkin()` position as a visual cross-check.
- Draws the camera settings window (virtual or real), then `update_scara_settings()`, then `imshow`.

Why clone? `cv::Mat` copies share their data. A plain `vector<Mat> box = _scara_links[i]` followed by `transformPoints()` wrote the result back into the stored model, so the arm was moved again every frame and disappeared off-screen. Cloning keeps the model fixed. (Lab 3 never hit this because its boxes are transformed only once.)

Part A-B: fkin(joints)

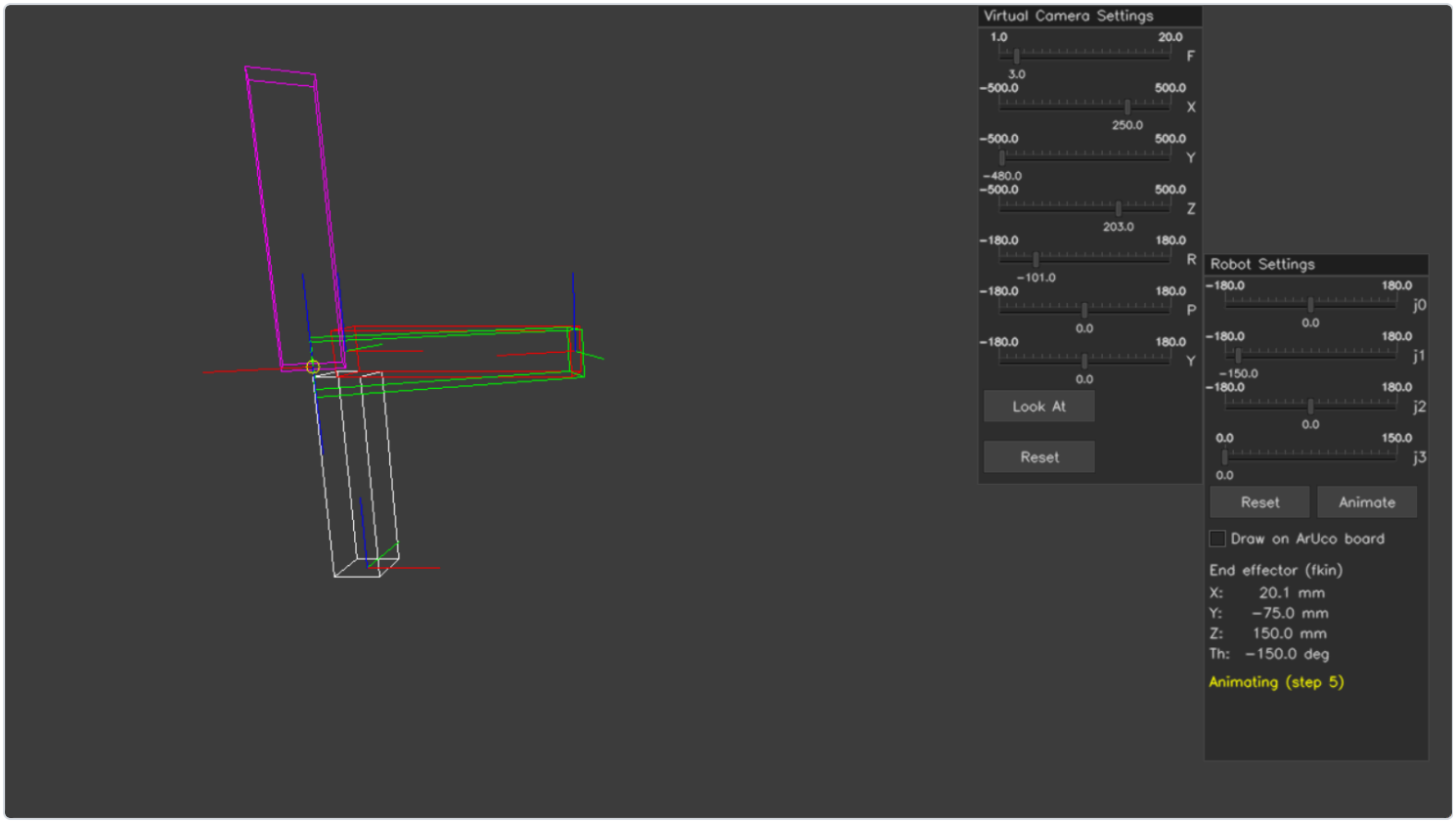
Input: (j0°, j1°, j2°, j3 metres). Output: Vec4d (x, y, z in metres, θ in degrees). It multiplies the same four `createHT` matrices straight through, without keeping intermediate poses, then reads the position from column 3. θ is the heading of the tool x axis: `atan2(T[1][0], T[0][0])`. For this SCARA that is equivalent to the closed form:

```
x = 0.15·cos(j0) + 0.15·cos(j0 + j1)
y = 0.15·sin(j0) + 0.15·sin(j0 + j1)
z = 0.15 − j3
θ = j0 + j1 + j2
```

Check from the test run: with j1 = −150°, the read-out showed X = 20.1 mm, Y = −75.0 mm, Th = −150°, which matches 150 + 150·cos(−150°) = 20.1 and 150·sin(−150°) = −75.0.

Part A-C: update_scara_settings(im)

Draws the "Robot Settings" window at the same place the template's `update_settings()` uses (right side, y = 220): four `cvui::trackbar` sliders bound to `_joint[0..3]`, Reset and Animate buttons, the board checkbox, and the fkin X / Y / Z / Th read-out (mm and degrees). It then calls `animate_scara()` and makes the **single** `cvui::update()` call for the frame, as the lab notes require for the buttons to work. `get_joints()` converts the slider ints to fkin units (j3 mm → m).



Mid-animation (state 5: j1 sweeping from +180 to −180). The fkin read-out and the yellow circle follow the tool.

6. Part B — functions

Part B-A: animate_scara()

This fills in the template's `_do_animate` state machine. Each frame moves one joint by `step_size` = 5 (degrees, or mm for j3) using a small `step_toward()` helper. The arm keeps redrawing while it moves, and the sliders show the live values.

State	Motion	Next
1 / 2 / 3	j0: 0 → +180, +180 → −180, −180 → 0	4
4 / 5 / 6	j1: 0 → +180, +180 → −180, −180 → 0	7
7	j2: one full turn 0 → 360. _anim_count counts to 360 and the slider value wraps past 180 to −180, because the slider is −180…180.	8
8	j3: 0 → 150 mm (15 cm)	9
9	Done: init() returns all joints to 0 (same as the template's last state)	0 (idle)

The whole sequence takes about 6 s on this PC. Change `step_size` in `animate_scara()` to slow it down.

Part B-B: drawing on the ArUco board

Ticking **Draw on ArUco board** sets `_draw_on_board`. `draw_scara()` then sets `_use_virtual_camera = false`, so the existing `drawBox()` / `drawCoord()` project through `_realcam` (solvePnP rvec/tvec + calibration) instead of the virtual camera. It then follows the same steps as `draw_simple_robot_lab4()`: get the image, `detect_markers()`, and paste the image at (0, 0). The robot is pre-multiplied by the Lab 4 board alignment, `createHT((0,0,0), (180,0,90))`, so it stands up off the board origin. Sliders, Reset and Animate all keep working because they only change `_joint[]`.

If no board pose has been found yet, the robot is skipped and a red message says so. Without a webcam, a "No webcam image" message appears instead. Testing: with the webcam on and no board in view, it showed the live image and the message without crashing. It has **not** been tested with a board in view.

7. How it fits together

```
lab5()
 ├─ CRobot robot              constructor: canvas, cvui, virtual cam, init()
 ├─ robot.create_scara()      Part A-A: boxes in joint frames, colours, camera view
 └─ loop until 'q'
     └─ robot.draw_scara()
         ├─ [board mode] _realcam.get_image / detect_markers         Part B-B
         ├─ scara_transforms(get_joints())  -> wT0..wT4              Part A-A
         ├─ drawBox / drawCoord for each box and frame
         ├─ fkin(get_joints())  -> yellow circle                     Part A-B
         ├─ _virtualcam.update_settings  or  _realcam.update_settings
         └─ update_scara_settings()                                  Part A-C
             ├─ sliders j0..j3, Reset, Animate, board checkbox
             ├─ fkin() pose read-out
             ├─ animate_scara()                                      Part B-A
             └─ cvui::update()   (only one per frame)
```

8. Notes and known limitations

- Real-camera mode is untested with an actual ChArUco board, because none was available. The code reuses the Lab 4 alignment, so if the robot appears rotated on the board, adjust the (180, 0, 90) in draw_scara().
- Already the case in Lab 4: CCameraReal::update_settings() draws its "Draw on Board" / "Draw all Markers" checkboxes at x = 5 on the left of the image, and they do nothing. Lab 5 uses its own checkbox in Robot Settings instead.
- Also from Lab 4: in board mode, get_image() and detect_markers() each grab a frame, so the pose comes from a frame slightly newer than the one shown.
- The Lab 6 controls in the PDF screenshot (X/Y/Z/Th sliders, Joint Control, Track Marker) were left out on purpose.
- Function name: create_scara() (all lower-case, matching create_simple_robot()).